ASSIGNMENT 2 – CO2

Q1 – Call by Value, Call by Reference and Call by Pointer

1. Aim

To understand the differences between call by value, call by reference and call by pointer in C++, and to implement a hostel fee concession updater.

2. Explanation

- Call by Value: A copy of the variable is passed. Changes do not affect the original variable.
- Call by Reference: The original variable is passed through a reference. Changes affect the original variable.
- Call by Pointer: The address of the variable is passed. Changes made using the pointer affect the original variable.

3. Program

```
#include <iostream>
using namespace std;

void change(int a, int& b, int* c)
{
   a += 10;
   b += 10;
   *c += 10;
}

int& bigger(int& p, int& q)
{
```

```
    return p > q ? p : q;
}

void show(int x, int y = 5, int z = 10)
{
    cout << x + y + z << ' ';
}

int main()
{
    int a = 1, b = 2, c = 3;

    change(a, b, &c);

    cout << a << ' ' << b << ' ' << c << '\n';

    bigger(a, c) = 100;

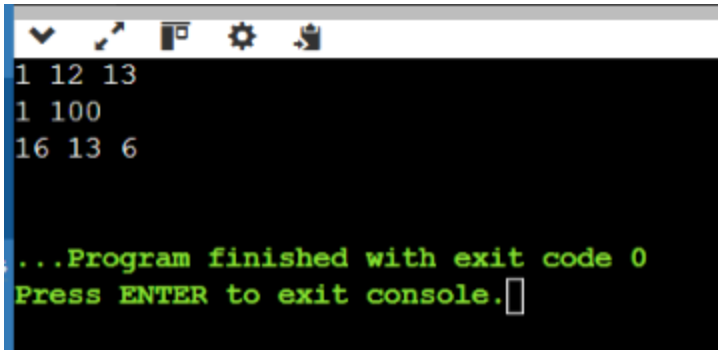
    cout << a << ' ' << c << '\n';

    show(1);
    show(1, 2);
    show(1, 2, 3);

    cout << '\n';

    return 0;
}
```

4. Output



5. Explanation of Output

- Variable a remains 1 because it is passed by value.
- Variable b becomes 12 because it is passed by reference.
- Variable c becomes 13 because it is passed by pointer.
- The bigger() function returns a reference to the larger variable. Therefore, assigning 100 changes c.
- show(1) uses default arguments 5 and 10.
- show(1,2) uses the default argument 10.
- show(1,2,3) uses all supplied arguments.

6. Complete the Code

```
#include <iostream>
using namespace std;

void applyByValue(long long fee, int percent)
{
   fee -= fee * percent / 100;
}

void applyConcession(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}
```

```
int main()
{
    long long hostelFee = 45000;

    applyByValue(hostelFee, 10);

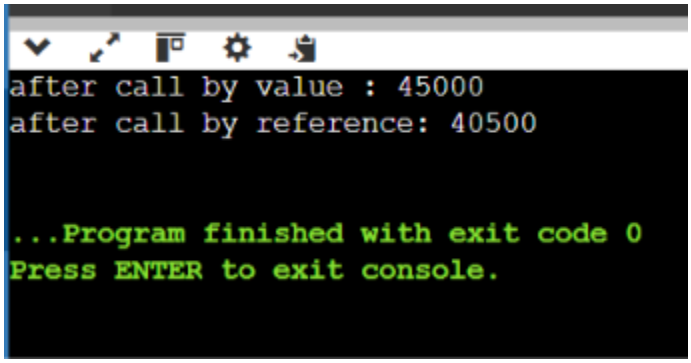
    cout << "after call by value : "
         << hostelFee << '\n';

    applyConcession(hostelFee, 10);

    cout << "after call by reference: "
         << hostelFee << '\n';

    return 0;
}
```

7. Output



8. Alternative Concept and Complexity Analysis

If the function returns the updated fee instead of modifying it through a reference, the program can be written as:

```
long long applyConcession(long long fee, int percent)
{
   return fee - fee * percent / 100;
}
```

The caller must store the returned value:

```
hostelFee = applyConcession(hostelFee, 10);
```

Method	**Time Complexity**	**Effect**
Call by value	O(1)	Original unchanged
Call by reference	O(1)	Original changed
Call by pointer	O(1)	Original changed
Call by const reference	O(1)	Read-only access

A const reference is useful when passing large objects that should not be modified because it avoids copying.

9. Conclusion

Call by reference and call by pointer are useful when a function needs to modify the original variable. For the hostel fee updater, call by reference is suitable because it directly updates the student's fee.

Q2 – Return by Reference and Its Dangers

1. Aim

To understand returning references from functions and the dangers of returning references to local variables.

2. Explanation

Returning by reference allows a function to return an existing variable without creating a copy. The caller can directly modify the original variable.

Returning a reference to a local variable is dangerous because the local variable is destroyed when the function ends. Accessing it through the returned reference causes undefined behaviour.

3. Program

```
#include <iostream>
using namespace std;

int& highestMark(int marks[], int n)
{
   int maxIndex = 0;

   for (int i = 1; i < n; i++)
   {
      if (marks[i] > marks[maxIndex])
      {
         maxIndex = i;
      }
   }

   return marks[maxIndex];
}

// Dangerous function: do not use its returned reference.
```

```
int& badReference()
{
   int x = 50;
   return x;
}

int main()
{
   int marks[5] = {75, 89, 92, 68, 85};

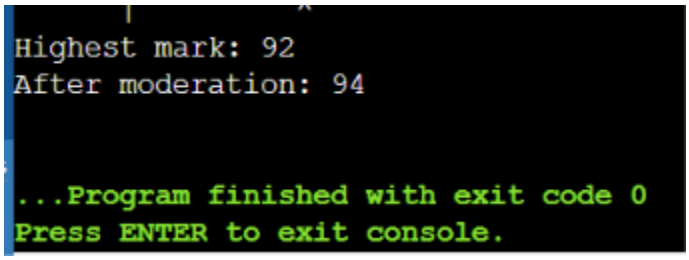
   cout << "Highest mark: "
        << highestMark(marks, 5) << endl;

   highestMark(marks, 5) += 2;

   cout << "After moderation: "
        << highestMark(marks, 5) << endl;

   return 0;
}
```

4. Output



5. Explanation

The highestMark() function finds the highest mark and returns a reference to its array element.

The statement:

highestMark(marks, 5) += 2;

directly increases the highest mark by 2.

The badReference() function returns a reference to a local variable. The compiler may produce a warning because the variable is destroyed when the function finishes.

6. Difference Between Return Types

Return type	Description
int&	Returns a modifiable reference
const int&	Returns a read-only reference
int	Returns a copy

7. Changed Condition: Vector

When an array is replaced by a std::vector, adding elements may cause reallocation. This can invalidate previously obtained references and pointers.

An index can be safer to retain across vector growth, provided the element remains at that index.

8. Alternative Concept and Complexity Analysis

The highest mark can also be returned as an index:

```
int highestIndex(int marks[], int n)
{
```

```
    int index = 0;

    for (int i = 1; i < n; i++)
    {
        if (marks[i] > marks[index])
            index = i;
    }

    return index;
}
```

Usage:

```
int index = highestIndex(marks, 5);
marks[index] += 2;
```

Method	**Time Complexity**	**Extra Space**
Reference	O(n)	O(1)
Index	O(n)	O(1)
Pointer	O(n)	O(1)

9. Conclusion

Returning by reference is useful when the caller needs to modify an existing variable. However, returning a reference to a local variable must be avoided because it causes undefined behaviour.

Q3 – Default Arguments and Function Overloading

1. Aim

To understand default arguments and function overloading in C++.

2. Explanation

Default arguments allow a function to use predefined values when the caller does not provide all arguments.

Function overloading allows multiple functions to have the same name but different parameter types or numbers.

The original courier functions create ambiguity because a call such as charge(2) can match more than one function.

3. Program

```
#include <iostream>
using namespace std;

struct Parcel
{
   double weight;
   bool express;
   bool insured;
};

// Weight in grams
void charge(int grams, bool express = false,
            bool insured = false)
{
   double amount = grams * 0.05;

   if (express) amount += 20;
```

```
    if (insured) amount += 10;

    cout << "Gram charge: " << amount << endl;
}

// Weight in kilograms
void charge(double kg, bool express = false,
            bool insured = false)
{
    double amount = kg * 50;

    if (express) amount += 20;
    if (insured) amount += 10;

    cout << "KG charge: " << amount << endl;
}

// Parcel object
void charge(const Parcel& p)
{
    double amount = p.weight * 50;

    if (p.express) amount += 20;
    if (p.insured) amount += 10;

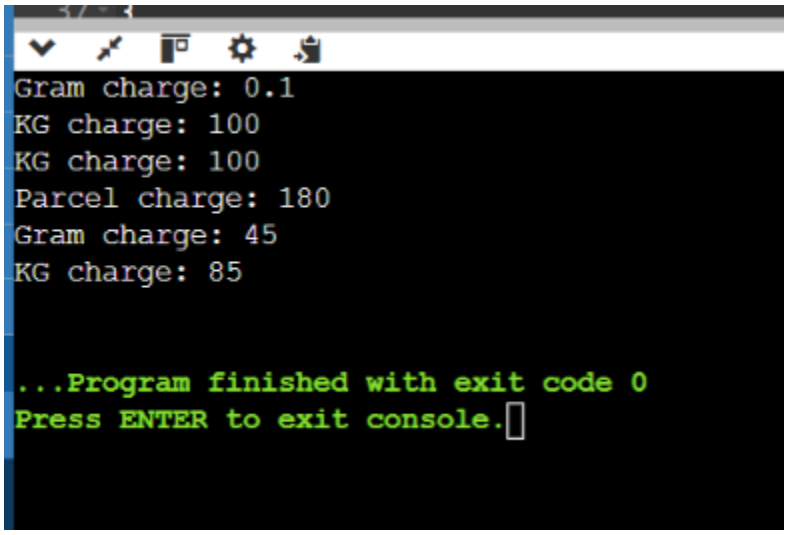
    cout << "Parcel charge: " << amount << endl;
}

int main()
{
    Parcel p = {3.0, true, true};
```

```
    charge(2);
    charge(2.0f);
    charge(2.0);
    charge(p);
    charge(500, true);
    charge(1.5, false, true);

    return 0;
}
```

4. Output



5. Explanation

1. charge(2) selects the integer version because the argument is an exact integer match.
2. charge(2.0f) selects the double version because float-to-double is a promotion.
3. charge(2.0) selects the double version because it is an exact match.
4. charge(p) selects the Parcel version.
5. charge(500, true) selects the integer version and uses the default value for insurance.
6. charge(1.5, false, true) selects the double version.

6. Rules of Default Arguments

Default arguments must be specified from right to left.

Correct:

void display(int a, int b = 10, int c = 20);

Incorrect:

void display(int a = 10, int b, int c = 20);

The compiler generally prefers an exact match over a promotion and a promotion over a conversion when selecting overloaded functions.

7. Alternative Concept and Complexity Analysis

Instead of multiple overloaded functions, a single structure can hold the parcel details.

```
struct Parcel
{
   double weight;
   bool express = false;
   bool insured = false;
};
```

A single function can calculate the charge using the structure.

Design	**Time Complexity**	**Overload Resolution**
Function overloading	O(1) calculation	Compile time

Structure-based function	O(1) calculation	Not required

Function overloading provides convenient function calls, while a structure makes adding new parcel options easier.

8. Conclusion

Default arguments reduce repeated code, and function overloading allows functions to handle different input types. A carefully designed set of overloaded functions avoids ambiguity.

Q4 – Specifying a Class with an Array Inside It

1. Aim

To create a SavingsAccount class using private data members, an array of transactions and public member functions.

2. Explanation

A class combines data and functions into a single unit.

The account number, balance, transaction array and transaction count are kept private to protect the account data.

The public functions provide controlled access to the account.

3. Program

```
#include <iostream>
using namespace std;

class SavingsAccount
{
private:
```

```
    int accountNumber;
    double balance;

    double transactions[10];
    int count;

public:
    SavingsAccount(int acc, double bal)
    {
        accountNumber = acc;
        balance = bal;
        count = 0;
    }

    void deposit(double amount)
    {
        if (amount <= 0) return;

        balance += amount;
        addTransaction(amount);
    }

    void withdraw(double amount)
    {
        if (amount <= 0 || amount > balance)
            return;

        balance -= amount;
        addTransaction(-amount);
    }
```

```
    void addTransaction(double amount);

    void miniStatement() const;
};

void SavingsAccount::addTransaction(double amount)
{
    if (count < 10)
    {
        transactions[count++] = amount;
    }
    else
    {
        for (int i = 1; i < 10; i++)
        {
            transactions[i - 1] = transactions[i];
        }

        transactions[9] = amount;
    }
}

void SavingsAccount::miniStatement() const
{
    cout << "Account Number: "
         << accountNumber << endl;

    cout << "Balance: " << balance << endl;

    cout << "Last Transactions:" << endl;
```

```
    for (int i = 0; i < count; i++)
    {
        cout << transactions[i] << endl;
    }
}

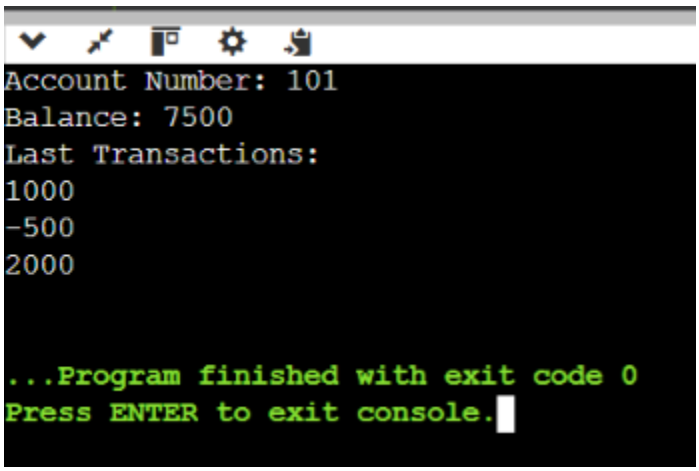
int main()
{
    SavingsAccount account(101, 5000);

    account.deposit(1000);
    account.withdraw(500);
    account.deposit(2000);

    account.miniStatement();

    return 0;
}
```

4. Output



5. Explanation

- The constructor initializes the account number and balance.
- deposit() increases the balance.
- withdraw() decreases the balance if sufficient money is available.
- addTransaction() stores the latest transactions.
- miniStatement() displays the account information.

The array is private because users should not directly modify transaction records.

6. Handling the 11th Transaction

When the array already contains 10 transactions, a new transaction arrives.

In the shifting method, the oldest transaction is removed, the remaining transactions are shifted left and the new transaction is added at the end.

This requires O(k) time for k transactions.

A circular buffer can instead use a start index to overwrite the oldest transaction in O(1) time.

7. Alternative Concept and Complexity Analysis

A std::deque can store transactions using:

```
transactions.push_back(amount);


if (transactions.size() > 10)
{
   transactions.pop_front();
}
```

Method	**Insertion Time**	**Space**
Fixed array with shifting	O(k)	O(k)

Circular buffer	O(1)	O(k)
Deque	O(1) at both ends, amortized	O(k)

8. Design Decision

For the requirement of storing the last 50 transactions, a circular buffer is suitable because its capacity can be changed from 10 to 50 without changing the basic algorithm.

9. Conclusion

A class with a private transaction array provides data security and controlled access. A circular buffer is more efficient than shifting elements when frequently adding transactions.

Q5 – Static Members and Arrays of Objects

1. Aim

To understand static data members, static member functions and arrays of objects in C++.

2. Explanation

A static data member belongs to the entire class rather than to an individual object.

All objects share the same static variable.

A static member function can access static members directly, but it cannot access non-static members without an object.

3. Predict the Output

```
#include <iostream>
using namespace std;
```

```
class Token
{
   static int next;
   int id;

public:
   Token()
   {
      id = ++next;
   }

   int get() const
   {
      return id;
   }

   static int issued()
   {
      return next - 100;
   }
};

int Token::next = 100;

int main()
{
   Token counter[3];

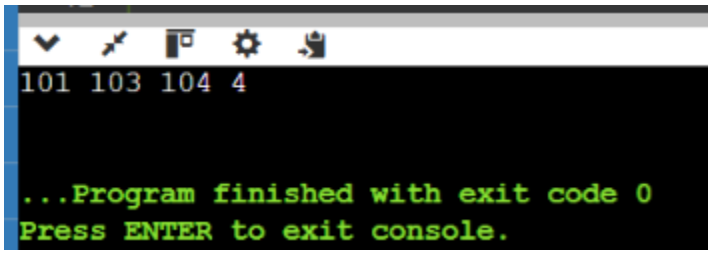
   Token extra;

   cout << counter[0].get() << ' '
```

```
        << counter[2].get() << ' '
        << extra.get() << ' '
        << Token::issued() << '\n';

    return 0;
}
```

4. Output



5. Explanation

The static variable next starts at 100.

Three objects are created in the array, and one additional object is created.

Therefore, the constructor executes four times.

The IDs are 101, 102, 103 and 104.

The issued() function returns 104 - 100 = 4.

6. Complete the Code

```
#include <iostream>
#include <string>
using namespace std;

class BusTicket
{
    static int issued;
```

```
    static const int CAPACITY = 3;

    int seat;
    string name;

public:

    explicit BusTicket(const string& n)
        : seat(0), name(n)
    {
        if (issued < CAPACITY)
        {
            issued++;
            seat = issued;
        }
    }

    static int count()
    {
        return issued;
    }

    void print() const
    {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat)
                      : string("Waiting"))
             << '\n';
    }
};
```

```
int BusTicket::issued = 0;

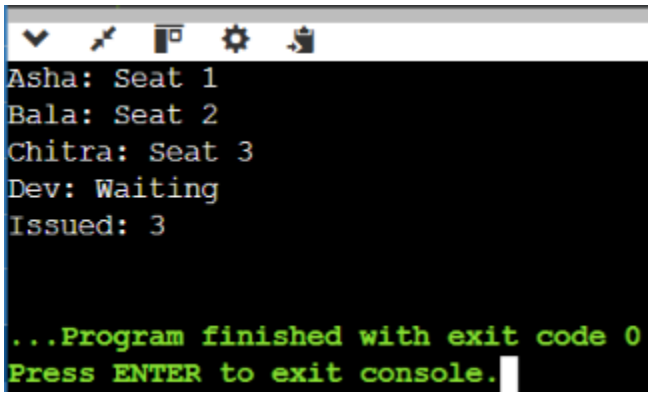
int main()
{
    BusTicket trip[4] = {
        BusTicket("Asha"),
        BusTicket("Bala"),
        BusTicket("Chitra"),
        BusTicket("Dev")
    };

    for (const BusTicket& t : trip)
    {
        t.print();
    }

    cout << "Issued: "
         << BusTicket::count() << '\n';

    return 0;
}
```

7. Output



8. Explanation

- The static variable issued is shared by all BusTicket objects.
- The capacity is fixed at three seats.
- Each new ticket receives the next seat number.
- The fourth ticket receives seat zero and is placed on the waiting list.
- The static function count() returns the total issued seats.

If tickets are created inside a function called twice, the static count continues from its previous value because it is shared by the class and is not reset when the function ends.

9. Alternative Concept and Complexity Analysis

A global variable could maintain the ticket count, but it would not provide good encapsulation.

A separate TicketOffice object could maintain the count and be passed to each ticket.

Design	**Time**	**Main Advantage**
Static member	O(1)	Shared class data
Global variable	O(1)	Simple implementation
TicketOffice object	O(1)	Supports multiple buses

A separate TicketOffice object is better when multiple buses need independent seat counters.

10. Conclusion

Static members are useful when data must be shared by all objects. Arrays of objects make it easy to create and manage multiple tickets.

Q6 – Friend Functions and Inline Functions

1. Aim

To understand friend functions and inline functions and implement a Time class to calculate the difference between two times.

2. Explanation

A friend function is not a member of a class, but it can access the private and protected members of that class.

An inline function is a function for which the compiler may substitute the function body at the point of the call to reduce function-call overhead.

3. Program

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time
{
   int h, m;

public:
   Time(int hh, int mm) : h(hh), m(mm) {}

   friend int minutesBetween(const Time& a,
                             const Time& b);

   friend bool earlier(const Time& a,
                       const Time& b);
};
```

```
inline int minutesBetween(const Time& a,
                          const Time& b)
{
   int t1 = a.h * 60 + a.m;
   int t2 = b.h * 60 + b.m;

   return abs(t1 - t2);
}

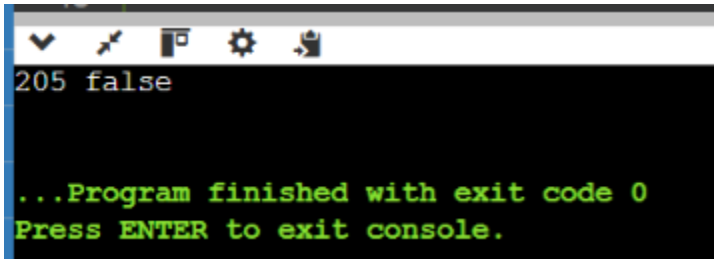
inline bool earlier(const Time& a,
                    const Time& b)
{
   return (a.h * 60 + a.m) <
          (b.h * 60 + b.m);
}

int main()
{
   Time departure(9, 45);
   Time arrival(13, 10);

   cout << minutesBetween(departure, arrival)
        << ' '
        << boolalpha
        << earlier(arrival, departure)
        << '\n';

   return 0;
}
```

4.Output



5. Explanation

The departure time is 9:45 AM and the arrival time is 1:10 PM.

Convert both times into minutes:

Departure:

9 × 60 + 45 = 585 minutes.

Arrival:

13 × 60 + 10 = 790 minutes.

Difference:

790 - 585 = 205 minutes.

The earlier() function checks whether the arrival time is earlier than the departure time. Therefore, the result is false.

6. Correctness Check

Test Case	**Expected Result**
9:45 and 13:10	205 minutes

10:00 and 10:00	0 minutes
11:30 and 12:30	60 minutes
14:00 and 12:00	120 minutes

7. Alternative Concept and Complexity Analysis

The same functionality can be implemented using member functions:

```
int minutesTo(const Time& b) const
{
   int t1 = h * 60 + m;
   int t2 = b.h * 60 + b.m;

   return abs(t1 - t2);
}

bool isEarlier(const Time& b) const
{
   return h * 60 + m < b.h * 60 + b.m;
}
```

Concept	**Time Complexity**	**Access**
Friend function	O(1)	Private members

Member function	O(1)	Private members
Public getters	O(1)	Public interface

Inline does not guarantee that the compiler will replace every function call with its body. The compiler makes the final decision.

8. Performance Investigation

The assignment requires testing 10,000,000 function calls with and without inline using -O0 and -O2.

Compilation commands:

g++ -std=c++17 -O0 program.cpp -o program
g++ -std=c++17 -O2 program.cpp -o program

Record the actual execution times:

Optimization	Without Inline	With Inline
-O0	_____ seconds	_____ seconds
-O2	_____ seconds	_____ seconds

The actual timing depends on the processor, compiler and optimization settings. It should be measured by executing both versions rather than estimated.

9. Conclusion

Friend functions are useful when an operation needs equal access to private data from multiple objects. Inline functions may reduce function-call overhead for small functions, although the compiler decides whether to inline them.

Q7 – Diagnose, Redesign and Validate

Selected Program: Q4 – Savings Account Mini-Statement

A. Diagnose the Limitation

The original SavingsAccount program uses a fixed array of 10 transactions.

When the 11th transaction arrives, the oldest transaction must be removed and the remaining elements must be shifted to the left.

This requires O(n) time for every new transaction when the array is full.

As the number of transactions increases, shifting creates unnecessary operations.

Identified limitation: The shifting method is inefficient for frequent transaction insertion.

B. Define the Design Objective

The objective is to improve transaction insertion performance.

The redesigned program should:

1. Store only the latest 10 transactions.
2. Remove the oldest transaction automatically.
3. Insert a new transaction in O(1) time.
4. Maintain the correct transaction order.
5. Use fixed memory without unnecessary shifting.

C. Design the Revised Solution

The redesigned solution uses a circular buffer.

A circular buffer is an array in which the last position connects back to the first position.

It uses a starting index and a count to maintain the transactions.

Changes made:

- The fixed array is retained.
- A starting index is introduced.
- New transactions overwrite the oldest transaction when the array becomes full.
- No shifting is required.
- The extra memory required is O(1).

Pseudocode:

Initialize start = 0
Initialize count = 0

When a transaction arrives:

If count is less than 10:
Insert at (start + count) % 10
Increase count

Otherwise:
Replace the element at start
Update start = (start + 1) % 10

Display transactions from start

D. Predict Before Implementing

The redesigned circular buffer is expected to insert a transaction in O(1) time instead of O(n) time.

The latest 10 transactions should always be available in the correct chronological order.

E. Implement and Validate the Redesign

C++ Program

```
#include <iostream>
using namespace std;

class SavingsAccount
{
private:
   int accountNumber;
   double balance;

   double transactions[10];
   int count;
   int start;

public:

   SavingsAccount(int acc, double bal)
      : accountNumber(acc), balance(bal),
        count(0), start(0) {}

   void deposit(double amount)
   {
      if (amount <= 0) return;

      balance += amount;
      addTransaction(amount);
   }
```

```
void withdraw(double amount)
{
   if (amount <= 0 || amount > balance)
      return;

   balance -= amount;
   addTransaction(-amount);
}

void addTransaction(double amount)
{
   if (count < 10)
   {
      int index = (start + count) % 10;
      transactions[index] = amount;
      count++;
   }
   else
   {
      transactions[start] = amount;
      start = (start + 1) % 10;
   }
}

void miniStatement() const
{
   cout << "Account Number: "
        << accountNumber << endl;

   cout << "Balance: "
```

```
            << balance << endl;

        cout << "Last Transactions:" << endl;

        for (int i = 0; i < count; i++)
        {
            int index = (start + i) % 10;
            cout << transactions[index] << endl;
        }
    }
};

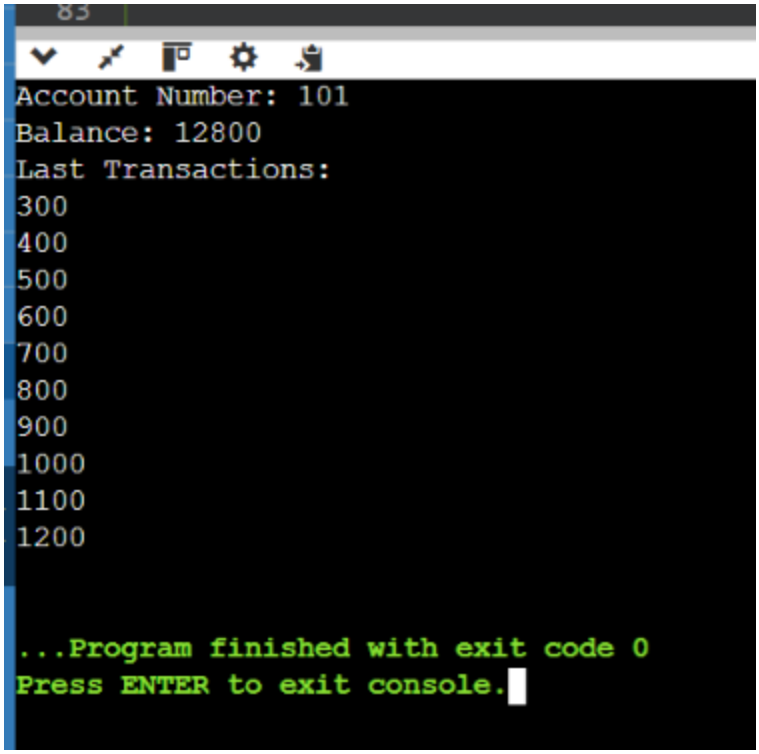
int main()
{
    SavingsAccount account(101, 5000);

    for (int i = 1; i <= 12; i++)
    {
        account.deposit(100 * i);
    }

    account.miniStatement();

    return 0;
}
```

Output



Comparison of Original and Redesigned Programs

Feature	**Original**	**Redesigned**
Data structure	Fixed array	Circular buffer
Insertion	O(n) when full	O(1)
Memory	O(n)	O(n)
Shifting	Required	Not required

Transaction order	Chronological	Chronological

F. Defend the Final Design

The redesigned circular buffer improves insertion efficiency because it does not shift the existing transactions.

It uses the same fixed amount of memory as the original array.

The original design is easier to understand and implement for a small number of transactions. However, the circular buffer is more suitable when transactions arrive frequently.

The objective is achieved at the algorithm-design level because insertion changes from O(n) to O(1).

Actual execution-time comparisons should be measured using the same compiler, machine and test data before reporting numerical improvements.

Final Conclusion

The circular buffer is a better design for managing the latest transactions because it provides constant-time insertion and avoids unnecessary shifting. It is suitable for applications such as banking systems, transaction records and real-time data processing.